Manual Técnico

Como desarrolladores para poder modificar código debemos clonar el repositorio de lo que queremos modificar, el enlace.

Para https: https://github.com/allan-mndz/Practica1_LFP.git

Para SSH: git@github.com:allan-mndz/Practica1_LFP.git

Sabemos que para clonar debemos situarnos en la carpeta en donde queremos clonar el repositorio y el comando a utilizar es: git clone <enlace>
Despues de tener clonado el repositorio abrimos el IDE en el que vamos a trabajar. Abrimos el IDE y le damos a

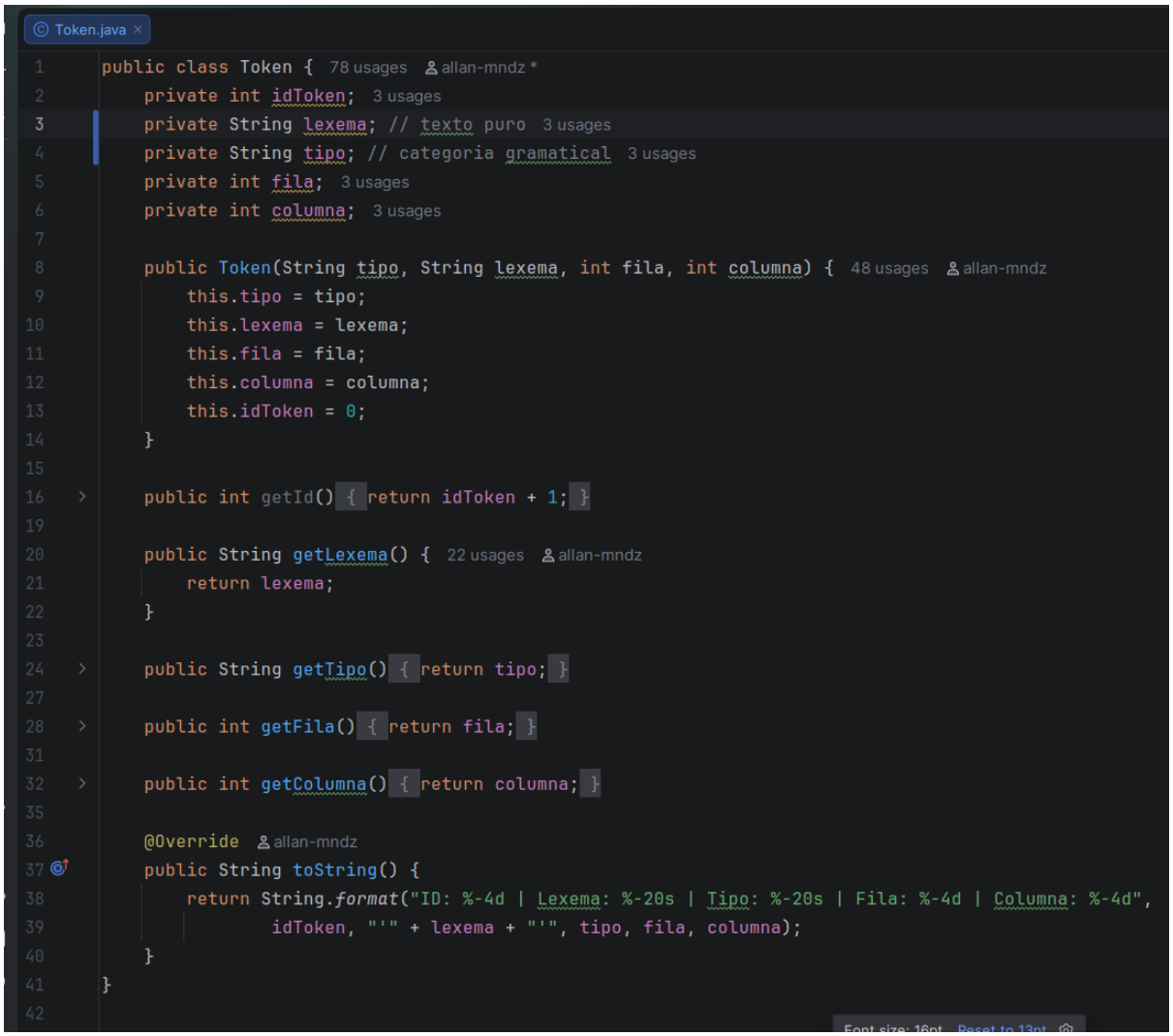
File > Open > carpeta donde clonamos > Practica1_LFP > abrir.

Y con eso ya tenemos el programa abierto listo para modificar lo que necesitamos, a continuación, la explicación de cada clase que se utilizó.

1. Análisis Léxico y Tokens

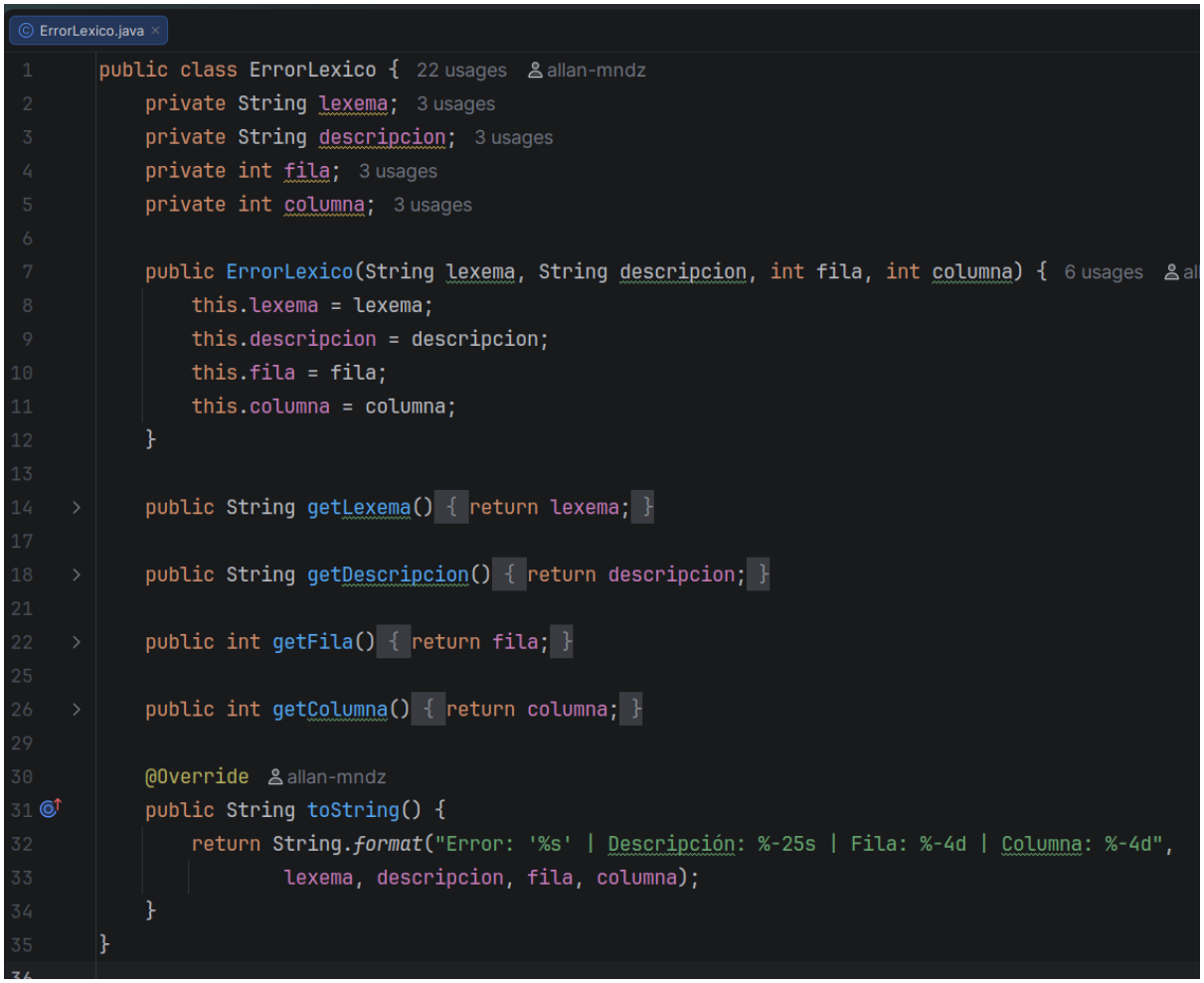
Token.java

Este objeto nos sirve para almacenar los atributos de una palabra reconocida y el método toString para imprimir los resultados finales ordenados.



ErrorLexico.java

Este objeto captura cualquier carácter extraño que no acepte el lenguaje PromptZal. En vez de que el programa se detenga, guarda el símbolo invalido y continua



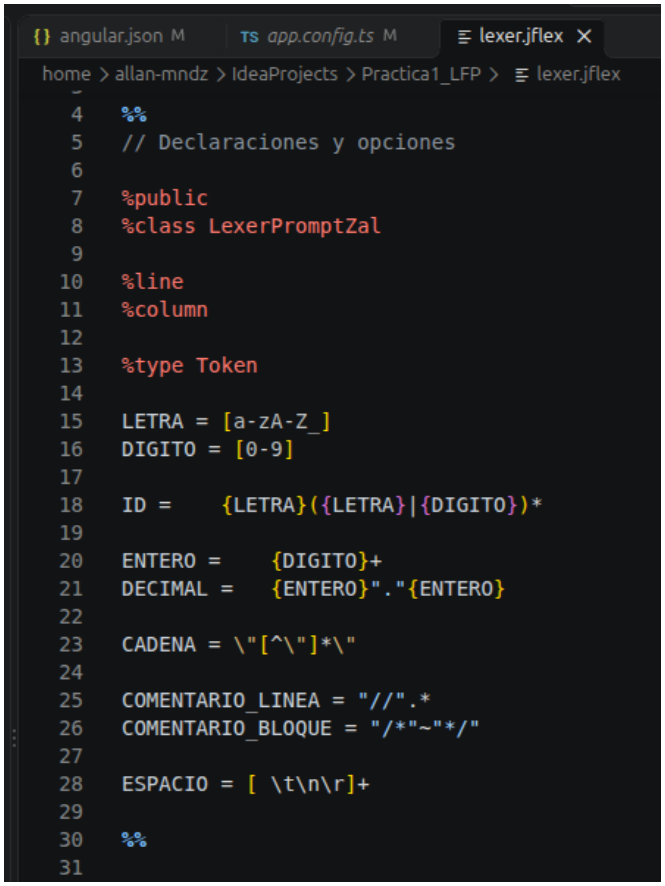
LexerPromptZal.jflex y LexerPromptZal.java

Esta clase es generada gracias al .jflex que realizamos en visual studio code

Para hacer que un .jflex se “transforme” a una clase .java es usando comandos de la terminal

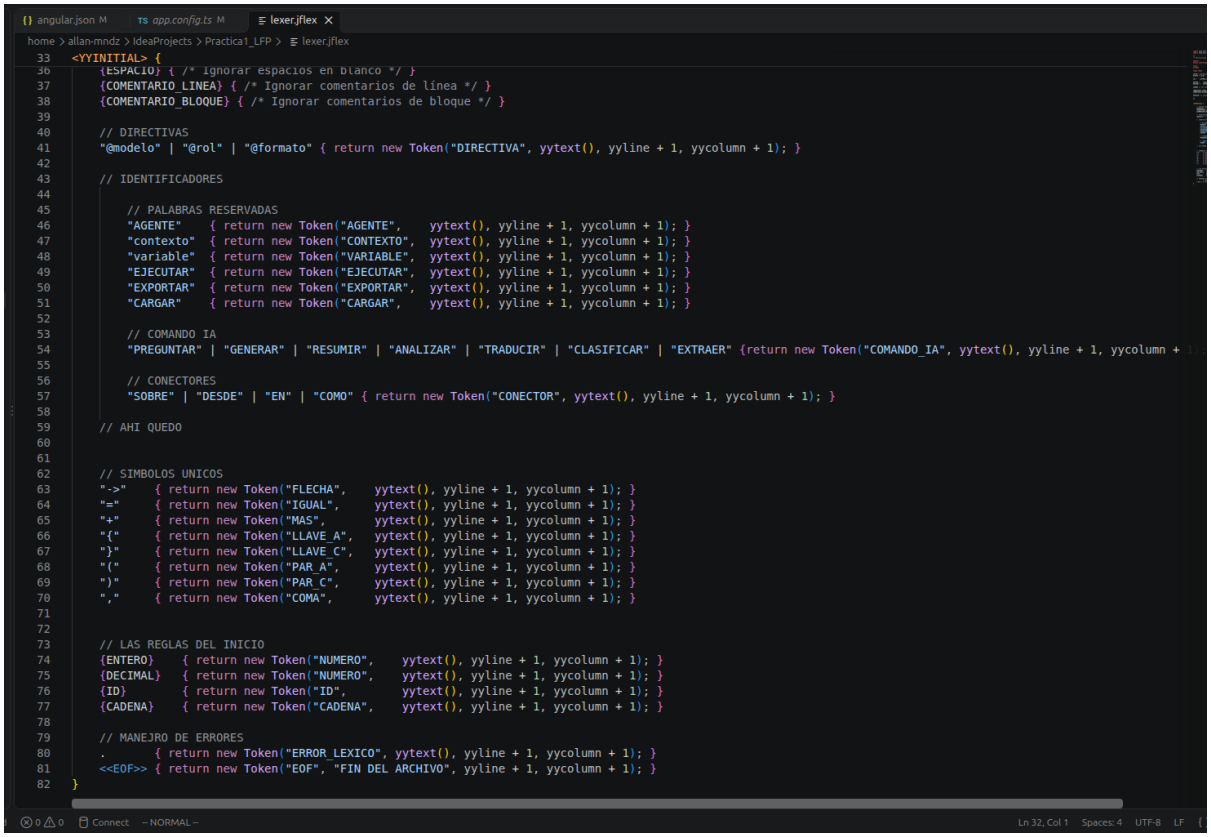
Pero primero debemos realizar el jflex para poderlo hacer.

En esta parte declaramos las importaciones que vamos a utilizar para hacer funcional nuestro analizador léxico



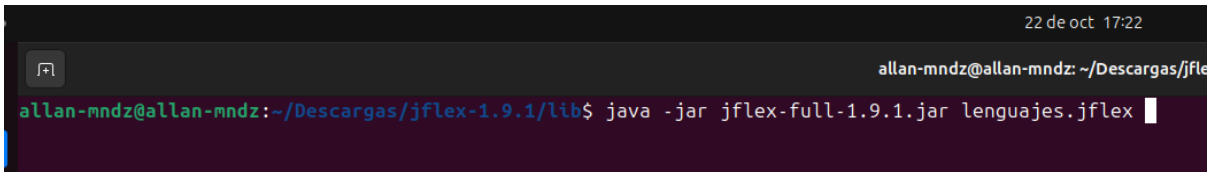
Después inicializamos todas nuestras reglas para hacer funcional o hacer que tenga sentido nuestro analizador.

Clasifica cadenas de texto en categorías especificas (reservadas, conectores, cadenas, números).



```
{} angular.json M   TS app.config.ts M   lexer.jflex ×
home > allan-mndz > IdeaProjects > Practica1_LFP > lexer.jflex
33  <YYINITIAL> {
36      {ESPACIO} { /* Ignorar espacios en blanco */ }
37      {COMENTARIO_LINEA} { /* Ignorar comentarios de línea */ }
38      {COMENTARIO_BLOQUE} { /* Ignorar comentarios de bloque */ }
39
40      // DIRECTIVAS
41      "@modelo" | "@rol" | "@formato" { return new Token("DIRECTIVA", yytext(), yyline + 1, yycolumn + 1); }
42
43      // IDENTIFICADORES
44
45          // PALABRAS RESERVADAS
46          "AGENTE"    { return new Token("AGENTE",    yytext(), yyline + 1, yycolumn + 1); }
47          "contexto"  { return new Token("CONTEXTO",  yytext(), yyline + 1, yycolumn + 1); }
48          "variable"  { return new Token("VARIABLE",  yytext(), yyline + 1, yycolumn + 1); }
49          "EJECUTAR"  { return new Token("EJECUTAR",  yytext(), yyline + 1, yycolumn + 1); }
50          "EXPORTAR"  { return new Token("EXPORTAR",  yytext(), yyline + 1, yycolumn + 1); }
51          "CARGAR"    { return new Token("CARGAR",    yytext(), yyline + 1, yycolumn + 1); }
52
53          // COMANDO IA
54          "PREGUNTAR" | "GENERAR" | "RESUMIR" | "ANALIZAR" | "TRADUCIR" | "CLASIFICAR" | "EXTRAER" {return new Token("COMANDO_IA", yytext(), yyline + 1, yycolumn +
55
56          // CONECTORES
57          "SOBRE" | "DESDE" | "EN" | "COMO" { return new Token("CONECTOR", yytext(), yyline + 1, yycolumn + 1); }
58
59      // AHI QUEDO
60
61
62      // SIMBOLOS UNICOS
63      "->"    { return new Token("FLECHA",    yytext(), yyline + 1, yycolumn + 1); }
64      "="     { return new Token("IGUAL",     yytext(), yyline + 1, yycolumn + 1); }
65      "+"     { return new Token("MAS",       yytext(), yyline + 1, yycolumn + 1); }
66      "{"     { return new Token("LLAVE_A",   yytext(), yyline + 1, yycolumn + 1); }
67      "}"     { return new Token("LLAVE_C",   yytext(), yyline + 1, yycolumn + 1); }
68      "("     { return new Token("PAR_A",     yytext(), yyline + 1, yycolumn + 1); }
69      ")"     { return new Token("PAR_C",     yytext(), yyline + 1, yycolumn + 1); }
70      ","     { return new Token("COMA",      yytext(), yyline + 1, yycolumn + 1); }
71
72
73      // LAS REGLAS DEL INICIO
74      {ENTERO}    { return new Token("NUMERO",    yytext(), yyline + 1, yycolumn + 1); }
75      {DECIMAL}   { return new Token("NUMERO",    yytext(), yyline + 1, yycolumn + 1); }
76      {ID}        { return new Token("ID",        yytext(), yyline + 1, yycolumn + 1); }
77      {CADENA}    { return new Token("CADENA",    yytext(), yyline + 1, yycolumn + 1); }
78
79      // MANEJRO DE ERRORES
80      .       { return new Token("ERROR_LEXICO", yytext(), yyline + 1, yycolumn + 1); }
81      <<EOF>> { return new Token("EOF", "FIN DEL ARCHIVO", yyline + 1, yycolumn + 1); }
82  }
⊗ 0 ⚠ 0   Connect   -- NORMAL --                                  Ln 32, Col 1   Spaces: 4   UTF-8   LF
```

Y por último vamos a la carpeta en donde tengamos el ejecutable (.jar) de jflex para realizar el siguiente comando y después de darle enter, nos creara él .java que usaremos en nuestro programa



```
22 de oct  17:22
allan-mndz@allan-mndz: ~/Descargas/jfle
allan-mndz@allan-mndz:~/Descargas/jflex-1.9.1/lib$ java -jar jflex-full-1.9.1.jar lenguajes.jflex
```

Tabla de Terminales

DIRECTIVA	@modelo, @rol, @formato
AGENTE CONTEXTO VARIABLE	AGENTE contexto variable
EJECUTAR	EJECUTAR
EXPORTAR	EXPORTAR
COMANDO_IA	PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER (y cualquier comando agregado)
CARGAR	CARGAR (es su propio terminal, NO es COMANDO_IA)
CONECTOR	SOBRE, DESDE, EN, COMO

FLECHA -> (NO es CONECTOR)

IGUAL	=
MAS	+
LLAVE_A / LLAVE_C	{ / }
PAR_A / PAR_C	(/)
COMA	,
ID	Identificadores (nombres de agentes y variables, y palabras como palabras)
CADENA	Texto entre comillas dobles
NUMERO	Enteros y decimales
EOF	Fin del archivo (lo genera el lexer al terminar)

2. Análisis Sintáctico y AST

Verificamos que la estructura cumpla con las reglas del lenguaje.

Gramática libre de contexto (BNF).

Usamos como referencia la gramática puesta en el proyecto. Se puede identificar que esta libre de recursividad por la izquierda para poderla implementar en el parser.

```
<programa> -> <directivas> <agentes> <ejecuciones> <exportar> EOF

<directivas> -> DIRECTIVA CADENA <directivas> | ε

<agentes> -> <agente> <mas_agentes>
<mas_agentes> -> <agente> <mas_agentes> | ε
<agente> -> AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C

<contexto> -> CONTEXTO IGUAL CADENA
<variables> -> VARIABLE ID IGUAL <expresion> <variables> | ε

<comandos> -> <comando> <mas_comandos>
<mas_comandos> -> <comando> <mas_comandos> | ε
<comando> -> COMANDO_IA <expresion> <conectores> FLECHA ID
<conectores> -> CONECTOR <expresion> <conectores> | ε

<expresion> -> <termino> <mas_terminos>
<mas_terminos> -> MAS <termino> <mas_terminos> | ε
<termino> -> CADENA | ID | NUMERO <unidad> | <cargar>
<unidad> -> ID | ε
<cargar> -> CARGAR PAR_A CADENA PAR_C

<ejecuciones> -> EJECUTAR ID <mas_ejecuciones>
<mas_ejecuciones> -> EJECUTAR ID <mas_ejecuciones> | ε

<exportar> -> EXPORTAR ID <mas_ids>
<mas_ids> -> COMA ID <mas_ids> | ε
```

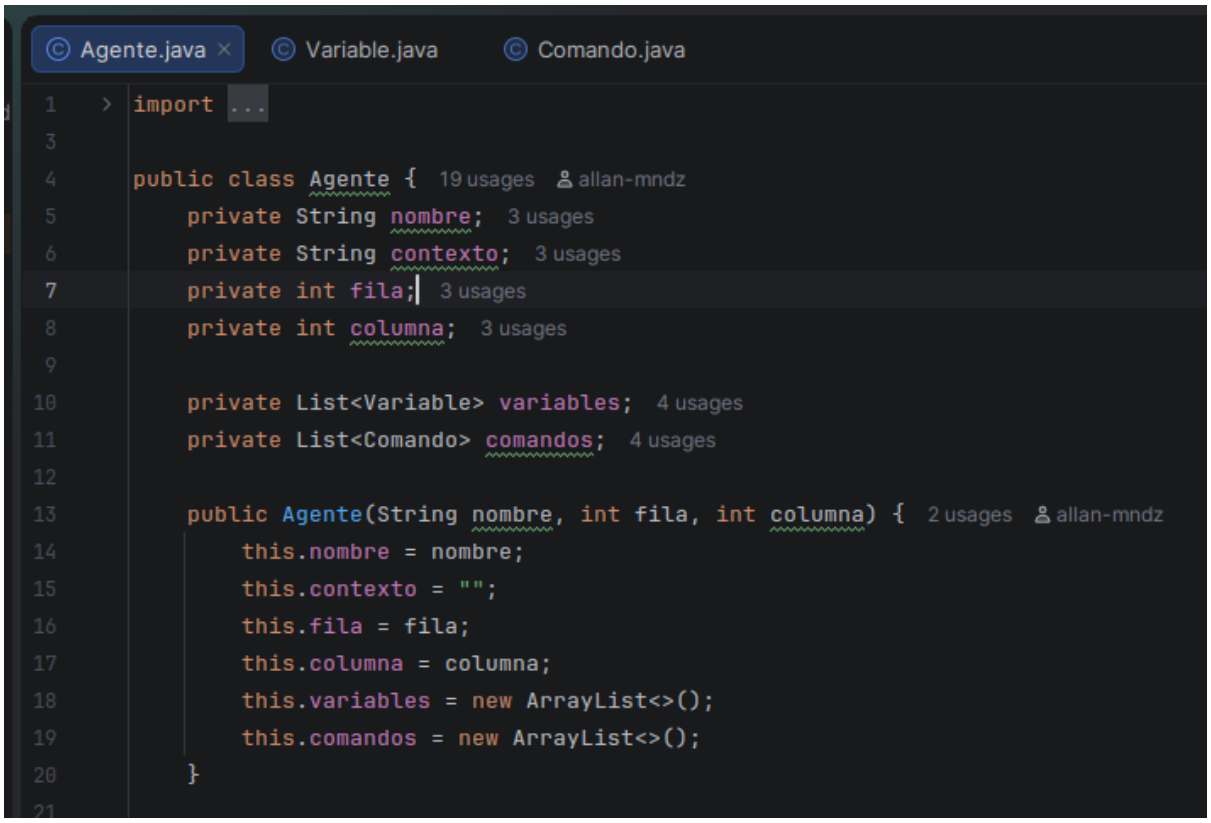
Diseño del AST

Agente.java

Es el nodo principal de cada bloque, almacena el nombre, contexto y administra dos listas dinámicas.

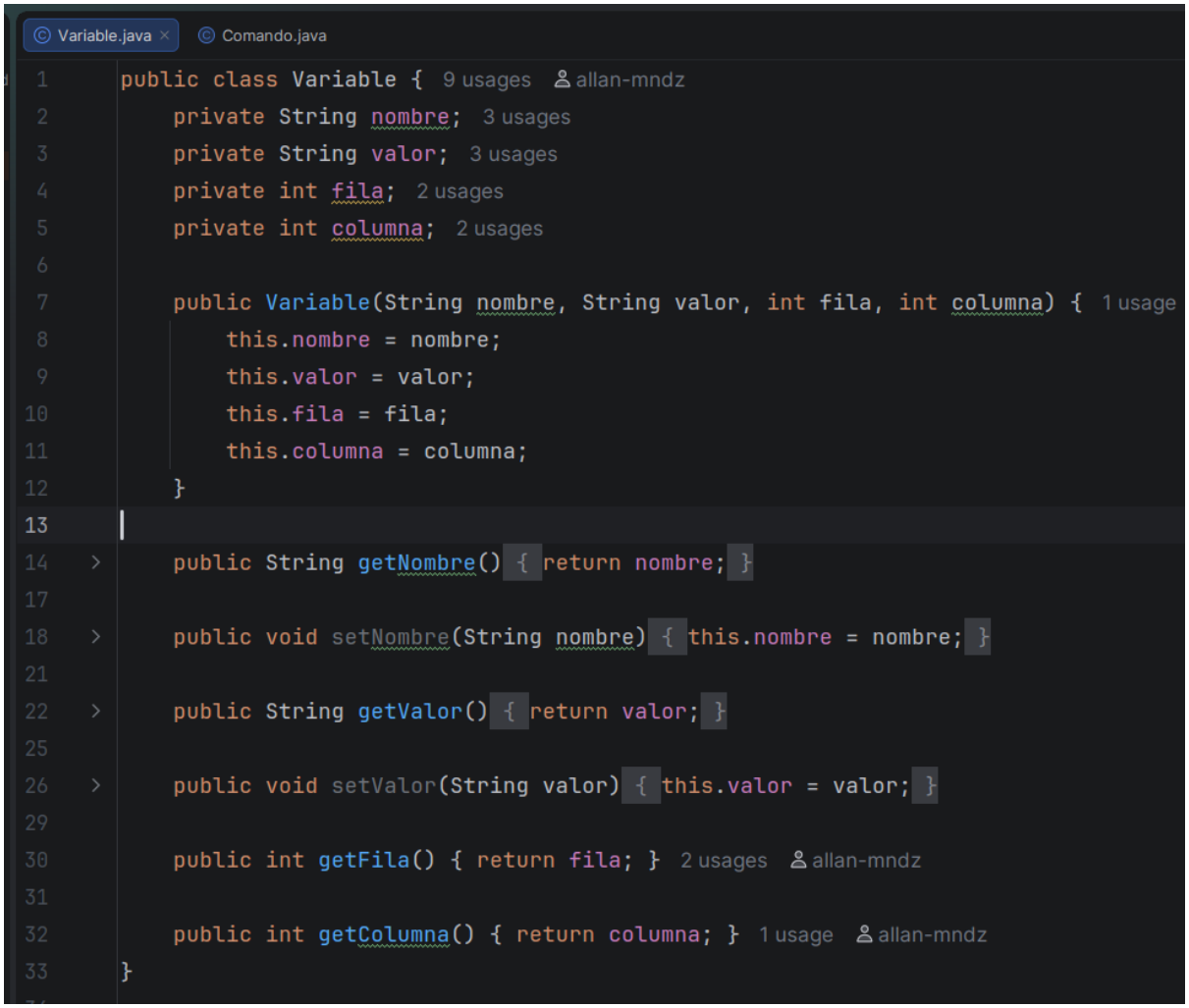
Una para las variables “List<Variable>”. Sirve para la fase semántica, ya que verifica si una variable existe antes de permitir que un comando la utilice.

Una para los comandos “List<Comando>”. Sirve para mantener el orden estricto de las instrucciones de IA, garantiza que las instrucciones se impriman secuencialmente en el mismo orden que fueron escritas.



Variable.java

Guarda el nombre y el valor que se le asigna, es decir el nombre que va antes del signo igual y el valor es el texto que va después de un signo igual.



Comando.java

Sirve para estructurar el desglose después de una instrucción de IA. Separa lógicamente el tipo de comando, el argumento principal, los conectores adicionales y la variable de destino donde se guardará el resultado.



Parser.java

Es un analizador sintáctico escrito a mano. Implementa un método específico para cada no terminal de la gramática.

```java
public class Parser {  2 usages  allan-mndz
    private int posicionActual;  4 usages
    private boolean modoPanico = false;  3 usages
    private Token tokenActual;  45 usages
    private List<Token> tokens;  3 usages
    private List<ErrorLexico> erroresSintacticos;  3 usages

    // cajas que sirven para guardar los datos utiles
    private List<Agente> listaAgentes;  3 usages
    private List<Token> ejecuciones;  4 usages
    private List<Token> exportaciones;  4 usages

    public Parser(List<Token> tokens) {  1 usage  allan-mndz
        this.tokens = tokens;
        this.erroresSintacticos = new ArrayList<>();
        this.listaAgentes = new ArrayList<>();
        this.ejecuciones = new ArrayList<>();
        this.exportaciones = new ArrayList<>();
        this.posicionActual = -1;
        avanzar();
    }
```

```java
private void avanzar(){  4 usages  allan-mndz
    posicionActual++;
    if (posicionActual < tokens.size()){
        tokenActual = tokens.get(posicionActual);
    }else{
        tokenActual = new Token( tipo: "EOF",  lexema: "FIN", fila: 0, columna: 0);
    }
}

private void match(String tipoEsperado){  35 usages  allan-mndz
    if (tokenActual.getTipo().equals(tipoEsperado)){
        modoPanico = false;
        avanzar();
    }else{
        if (modoPanico == false){
            String descripcionError = String.format("Se esperaba '%s' pero encontro '%s'", tipoEsperado, tokenActual.getTipo());

            ErrorLexico nuevoError = new ErrorLexico(tokenActual.getLexema(), descripcionError, tokenActual.getFila(), tokenActual.getColumna());

            erroresSintacticos.add(nuevoError);
            modoPanico = true;
        }
        recuperacionModoPanico();
    }
}
```

Cuando se detecta un token que no coincide con la producción esperada, el parser no se detiene. Reporta el error sintáctico detallando el token encontrado y el esperado, y activa el modo pánico. Cuando está en el modo pánico descarta tokens de la

entrada hasta encontrar un token de sincronización seguro, los cuales serían.



```
52      private void recuperacionModoPanico(){ 1 usage  allan-mndz
53          if (esTokenValido(tokenActual.getTipo()) || tokenActual.getTipo().equals("EOF")){
54              return;
55          }
56          // dejamos los errores atras y avanzamos
57          avanzar();
58          while (!esTokenValido(tokenActual.getTipo()) && !tokenActual.getTipo().equals("EOF")) {
59              avanzar();
60          }
61      }
62
63  @   private boolean esTokenValido(String tipo) { 2 usages  allan-mndz
64          return tipo.equals("AGENTE") ||
65                  tipo.equals("LLAVE_C") ||
66                  tipo.equals("COMANDO_IA") ||
67                  tipo.equals("VARIABLE") ||
68                  tipo.equals("EJECUTAR") ||
69                  tipo.equals("EXPORTAR");
70      }
```

Esto permite continuar con el análisis.

3. Análisis Semántico y Tabla de Símbolos

Aqui recibimos el AST que el parser construyo y se encarga de validar el significado del programa. Es decir que revisa que se cumplan las reglas antes de generar el código final.

Simbolo.java

Esta clase representa una única entidad individual. Cada vez que compilamos y detecte un agente o una variable nueva en el código, crea un objeto con los atributos de la clase para guardarlo.

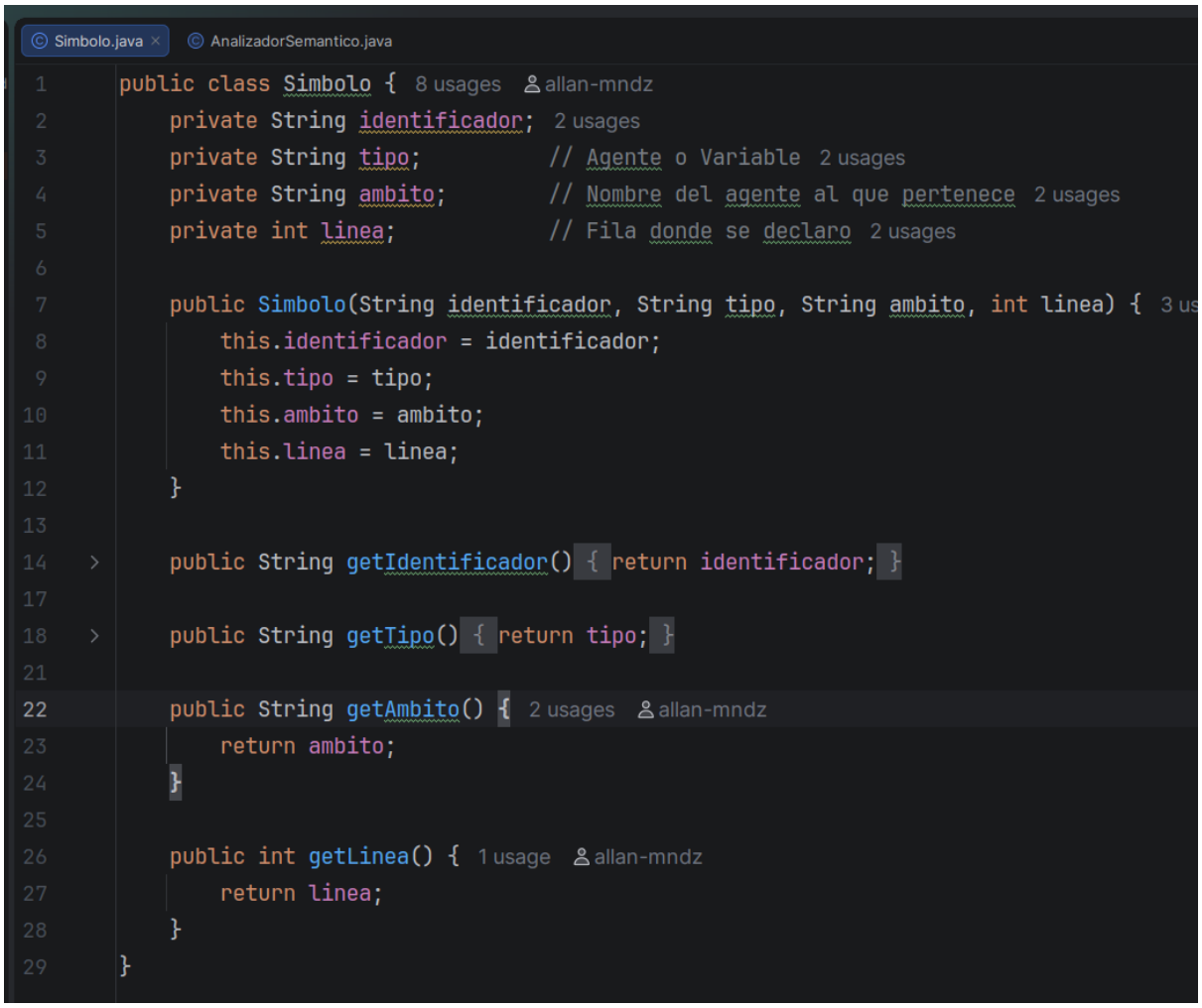


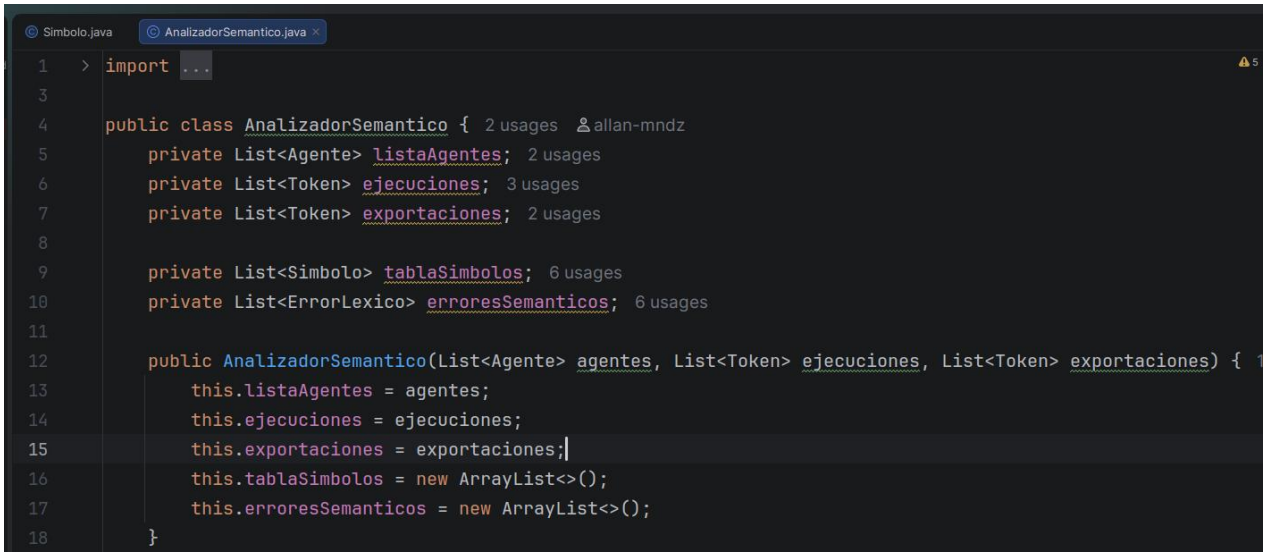
Tabla de Símbolos

La tabla de símbolos es la lista en memoria que va guardando todos los objetos símbolos a medida que el analizador semántico recorre el AST. Antes que un comando IA procese una variable, el sistema consulta a la lista, para asegurarse de que la variable haya sido declarada previamente, así damos por hecho que las variables de un agente sean invisibles para los demás. Todo esto se maneja en la clase del Analizador Semántico.

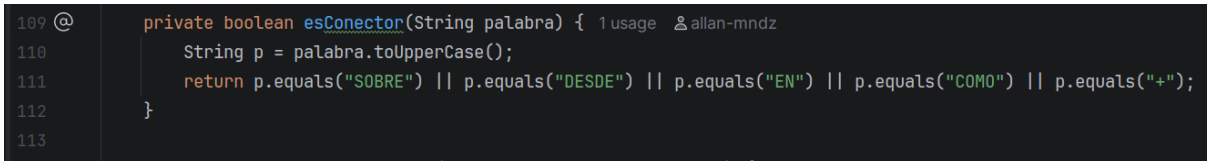
AnalizadorSemantico.java

Esta clase nos sirve para la validación lógica del programa. Su responsabilidad es garantizar que el árbol construido por el parser tenga sentido y cumpla con las reglas del lenguaje.

Necesitamos tener las 3 listas extraídas del AST por el parser e inicializa dos listas vacías una que son tablaSimbolos (lo que explicamos) y erroresSemanticos (para guardar cualquier fallo sin detener el proceso).



Un método booleano que identifica palabras reservadas para que el validador profundo las ignore al buscar variables.



Recorre la tabla de símbolos y retorna true solo si encuentra una coincidencia exacta de nombre y ámbito.

```java
    private boolean existeSimbolo(String nombre, String ambito) {  4 usages  allan-mndz
        for (Simbolo s : tablaSimbolos) {
            if (s.getIdentificador().equals(nombre) && s.getAmbito().equals(ambito)) {
                return true;
            }
        }
        return false;
    }

```

El método recorre el AST en orden y aplica las validaciones obligatorias.

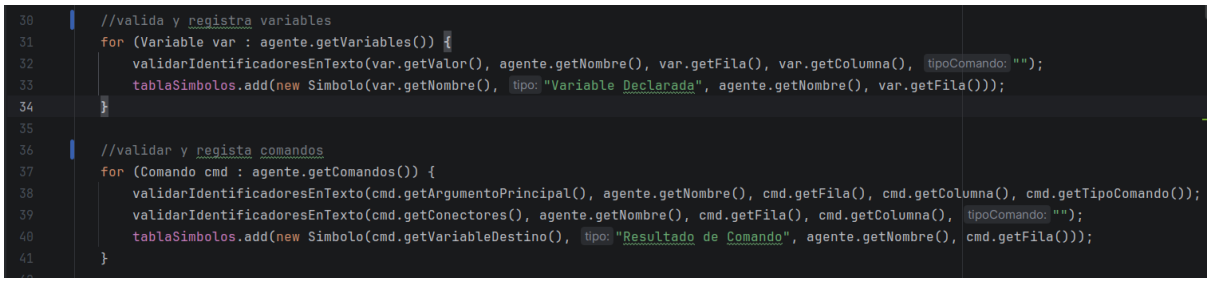
Por cada Agente, verifica si ya existe en el ámbito “Global”, es decir que si ya se utilizó un tipo de agente una vez reporta un error de duplicidad, sino es así, lo registra en la tabla de símbolos.



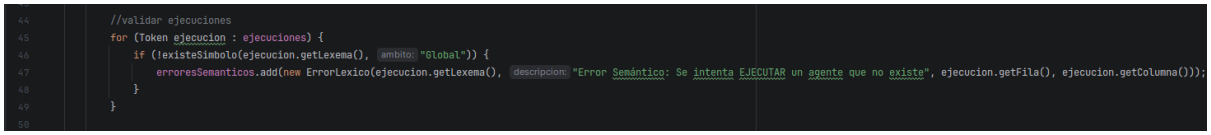
Recorre las variables y comandos del agente.

Verifica que las variables usadas en la expresión ya existan y guarda esta nueva variable en la tabla de símbolos.

Verifica las variables usadas en el argumento principal del comando, también verifica las variables usadas en los conectores y guarda la variable destino en la tabla.



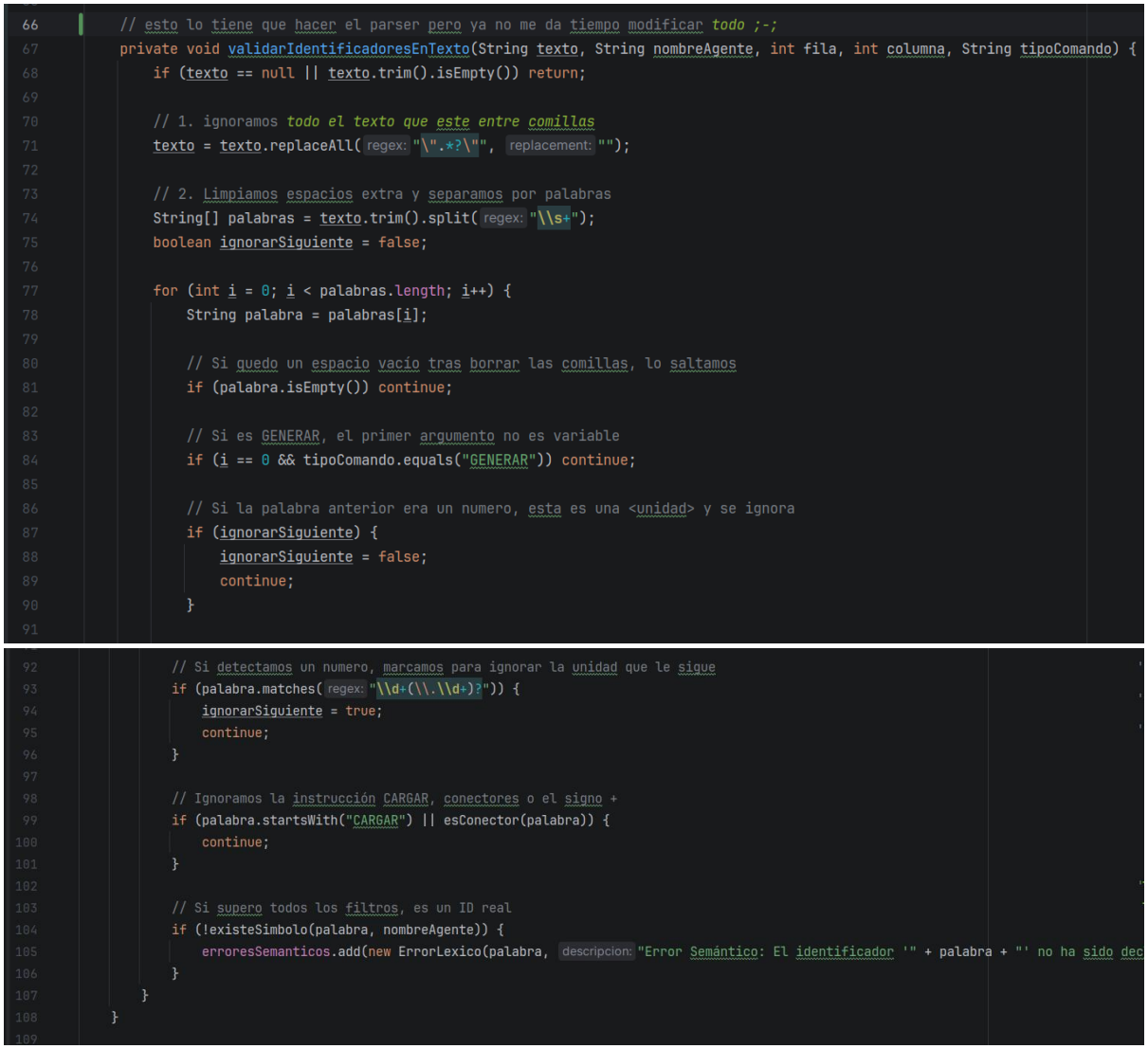
Recorre la lista Ejecutar y verifica en la tabla de símbolos que el identificador llamado pertenezca a un Agente que realmente exista.



Recorre la lista Exportar y se asegura de que la variable solicitada exista claramente dentro de los agentes que fueron marcados para ejecutarse.



Dado que el parser almacena las expresiones matemáticas o de IA como un solo texto concatenado, este método actúa como un "mini-lexer" para cumplir la regla de que el identificador debe estar declarado antes de usarlo.



```java
// esto lo tiene que hacer el parser pero ya no me da tiempo modificar todo ;-;
private void validarIdentificadoresEnTexto(String texto, String nombreAgente, int fila, int columna, String tipoComando) {
    if (texto == null || texto.trim().isEmpty()) return;

    // 1. ignoramos todo el texto que este entre comillas
    texto = texto.replaceAll( regex: "\".*?\"", replacement: "");

    // 2. Limpiamos espacios extra y separamos por palabras
    String[] palabras = texto.trim().split( regex: "\\s+");
    boolean ignorarSiguiente = false;

    for (int i = 0; i < palabras.length; i++) {
        String palabra = palabras[i];

        // Si quedo un espacio vacío tras borrar las comillas, lo saltamos
        if (palabra.isEmpty()) continue;

        // Si es GENERAR, el primer argumento no es variable
        if (i == 0 && tipoComando.equals("GENERAR")) continue;

        // Si la palabra anterior era un numero, esta es una <unidad> y se ignora
        if (ignorarSiguiente) {
            ignorarSiguiente = false;
            continue;
        }

        // Si detectamos un numero, marcamos para ignorar la unidad que le sigue
        if (palabra.matches( regex: "\\d+(\\.\\d+)?")) {
            ignorarSiguiente = true;
            continue;
        }

        // Ignoramos la instrucción CARGAR, conectores o el signo +
        if (palabra.startsWith("CARGAR") || esConector(palabra)) {
            continue;
        }

        // Si supero todos los filtros, es un ID real
        if (!existeSimbolo(palabra, nombreAgente)) {
            erroresSemanticos.add(new ErrorLexico(palabra, descripcion: "Error Semántico: El identificador '" + palabra + "' no ha sido dec
        }
    }
}
```

4. Generar Prompt y Swing

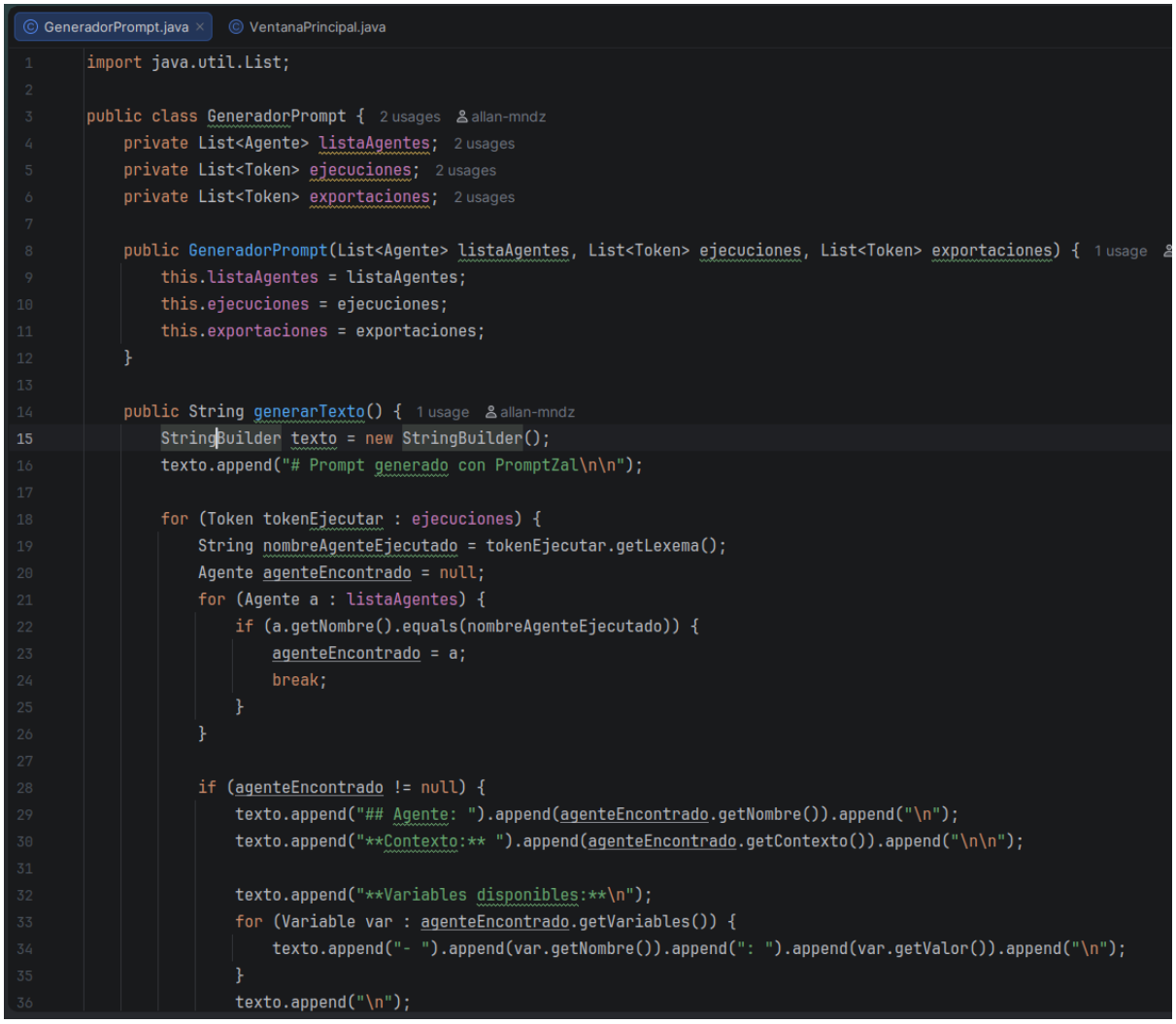
GeneradorPrompt.java

Esta clase se encarga de transformar la representación en memoria del programa hacia su formato final.

El método generarTexto funciona de la siguiente manera:

Creamos el StringBuilder con el título principal. Luego hacemos un for para la lista de ejecuciones y buscamos el nodo correspondiente dentro de la lista global de agentes construidas en el AST.

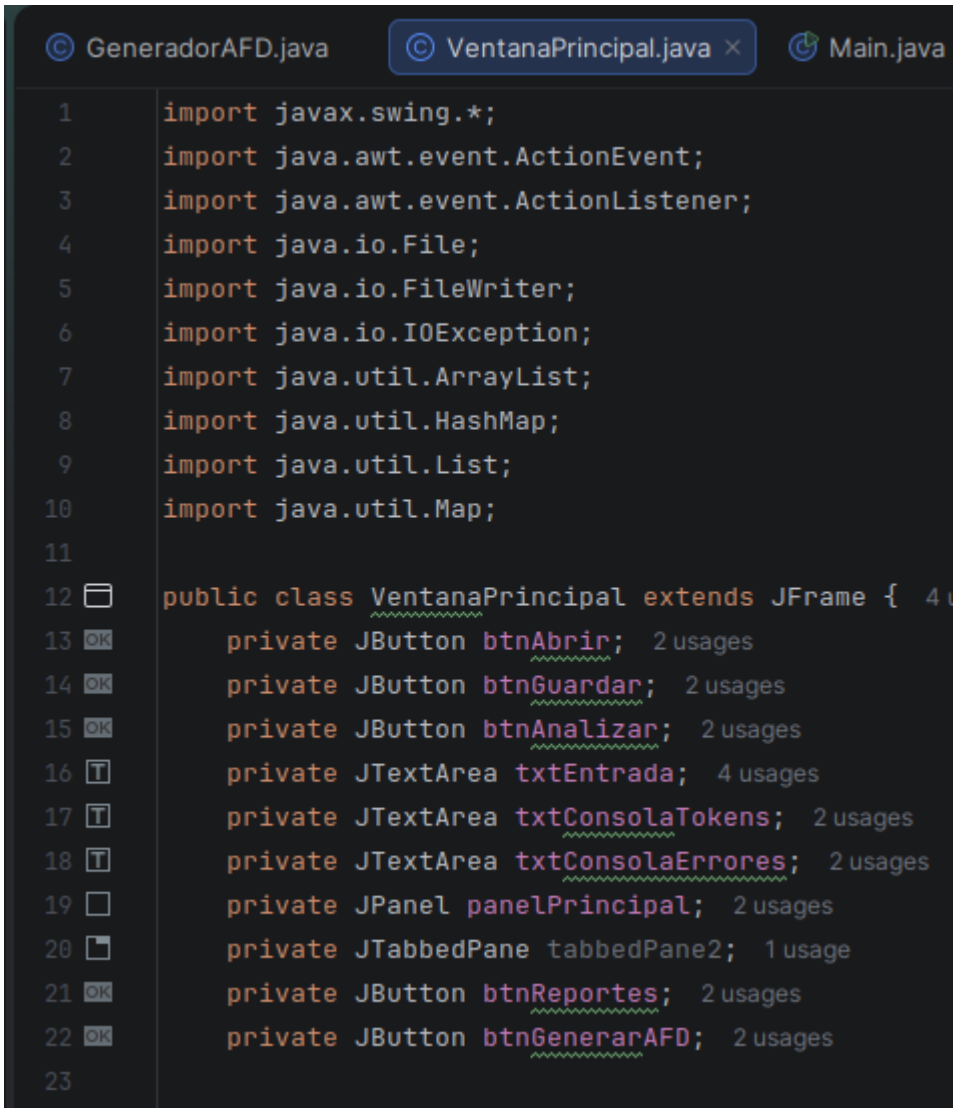
Al confirmar la existencia del agente, concatena su nombre y su contexto con formato de subtítulos ## y **. Después recorre la sub-lista interna de variables del agente para imprimirlas con viñetas, mostrando el identificador y la expresión asignada.



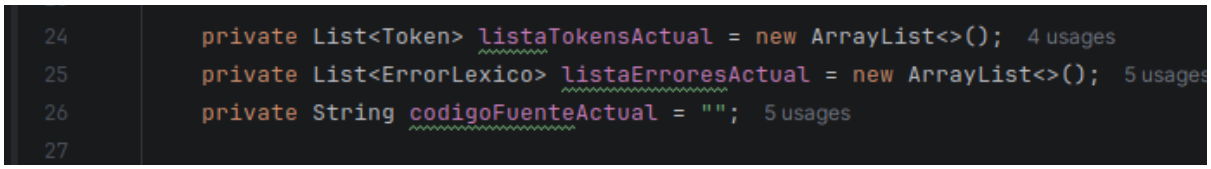
VentanaPrincipal.java

En la ventana principal extiende de JFrame porque es una ventana de swing para tener una vista más cómoda para trabajar.

Esta ventana está compuesta por los siguientes elementos



También instanciamos estas variables que vamos a usar dentro de la clase



Botón Analizar

Primero verificamos que en la caja de texto no este vacío y luego llamamos a nuestra clase Analizador Léxico y nuestro método analizar

```
btnAnalizar.addActionListener(new ActionListener() {  allan-mndz *
    @Override  no usages  allan-mndz *
    public void actionPerformed(ActionEvent e) {
        lexema = txtEntrada.getText();

        if (lexema.trim().isEmpty()) { // verifica si no hay texto o solo tiene espacions
            JOptionPane.showMessageDialog( parentComponent: null,  message: "El editor está vacio.",  title: "Advertencia", JOptionPane.WARNING_MESSAGE);
            return;
        }

        AnalizadorLexico analizador = new AnalizadorLexico(lexema);
        analizador.analizar();
```

Botón Guardar

Con este botón podemos guardar el archivo que estemos modificando o creando dentó de la caja de texto en donde se analiza el archivo .pz todo esto es gracias al uso de JFileChooser y con una condición que el archivo debe tener la ruta .pz

```
82          btnGuardar.addActionListener(new ActionListener() {  allan-mndz
83              @Override  no usages  allan-mndz
84              public void actionPerformed(ActionEvent actionEvent) {
85                  JFileChooser guardar = new  JFileChooser();
86                  int archivoGuardado = guardar.showSaveDialog( parent: null);
87
88                  if (archivoGuardado == JFileChooser.APPROVE_OPTION) {
89                      File archivo = guardar.getSelectedFile();
90
91                      String ruta = archivo.getPath();
92                      if(!ruta.endsWith(".pz")){
93                          archivo = new File( pathname: ruta + ".pz");
94                      }
95
96                      try(FileWriter fw = new FileWriter(archivo)){
97                          fw.write(txtEntrada.getText());
98                          JOptionPane.showMessageDialog( parentComponent: null,  message: "Archivo .pz guardado con exito");
99                      }catch (IOException e){
100                         JOptionPane.showMessageDialog( parentComponent: null,  message: "Error al guardar el archivo");
101                     }
102
103                 }
104             }
105         });
106
107
```

Botón Abrir

Este botón nos sirve para abrir explorador de archivos de la computadora y buscar un archivo para poder analizar y detectar sus tokens y sus errores, esto gracias también al JFileChooser

```
109         btnAbrir.addActionListener(new ActionListener() {  allan-mndz *
110             @Override  no usages  allan-mndz *
111             public void actionPerformed(ActionEvent actionEvent) {
112                 JFileChooser explorador = new JFileChooser();
113                 int resultado = explorador.showOpenDialog( parent: null);
114
115                 if (resultado == JFileChooser.APPROVE_OPTION) {
116                     File archivoSeleccionado = explorador.getSelectedFile();
117                     try {
118                         String contenido = Files.readString(archivoSeleccionado.toPath());
119                         txtEntrada.setText(contenido);
120                     }catch (Exception ex){
121                         JOptionPane.showMessageDialog( parentComponent: null, ex.getMessage());
122                     }
123                 }
124             }
125         });
```

Botón Reportes

Primero se tiene que analizar un documento .pz para poder generar los reportes, después de analizarlo, nuevamente tenemos que elegir donde queremos guardar los reportes

```
127     btnReportes.addActionListener(new ActionListener() {  allan-mndz *
128         @Override  no usages  allan-mndz *
129         public void actionPerformed(ActionEvent actionEvent) {
130             if (lexema.isEmpty() && listaTokensActual.isEmpty() && listaErroresActual.isEmpty()) {
131                 JOptionPane.showMessageDialog( parentComponent: null,  message: "Primero debes Analizar un documento .pz para poder generar reportes",  title: "Advertencia", JOpt
132                 return;
133             }
134
135             // Abrimos el explorador para elegir la carpeta destino
136             JFileChooser explorador = new JFileChooser();
137             explorador.setDialogTitle("Selecciona la carpeta para guardar los reportes");
138             explorador.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);
139
140             int seleccion = explorador.showSaveDialog( parent: null);
141
142             if (seleccion == JFileChooser.APPROVE_OPTION) {
143                 // Obtenemos la ruta de la carpeta elegida
144                 String rutaCarpeta = explorador.getSelectedFile().getAbsolutePath();
145
146                 // Llamamos al metodo pasandole la nueva ruta
147                 generarReportesHTML(listaTokensActual, listaErroresActual, lexema, rutaCarpeta);
148             }
149         }
150     });
```